

orthogonal concern.

A key benefit of the “package by component” approach is that if you’re writing code that needs to do something with `orders`, there’s just one place to go—the `OrdersComponent`. Inside the component, the separation of concerns is still maintained, so the business logic is separate from data persistence, but that’s a component implementation detail that consumers don’t need to know about. This is akin to what you might end up with if you adopted a micro-services or Service-Oriented Architecture—a separate `OrdersService` that encapsulates everything related to handling orders. The key difference is the decoupling mode. You can think of well-defined components in a monolithic application as being a stepping stone to a micro-services architecture.

THE DEVIL IS IN THE IMPLEMENTATION DETAILS

On the face of it, the four approaches do all look like different ways to organize code and, therefore, could be considered different architectural styles. This perception starts to unravel very quickly if you get the implementation details wrong, though.

Something I see on a regular basis is an overly liberal use of the `public` access modifier in languages such as Java. It’s almost as if we, as developers, instinctively use the `public` keyword without thinking. It’s in our muscle memory. If you don’t believe me, take a look at the code samples for books, tutorials, and open source frameworks on GitHub. This tendency is apparent, regardless of which architectural style a code base is aiming to adopt—horizontal layers, vertical layers, ports and adapters, or something else. Marking all of your types as `public` means you’re not taking advantage of the facilities that your programming language provides with regard to encapsulation. In some cases, there’s literally nothing preventing somebody from writing some code to instantiate a concrete implementation class directly, violating the intended architecture style.

ORGANIZATION VERSUS ENCAPSULATION

Looking at this issue another way, if you make all types in your Java application `public`, the packages are simply an organization mechanism (a grouping, like

folders), rather than being used for encapsulation. Since public types can be used from anywhere in a code base, you can effectively ignore the packages because they provide very little real value. The net result is that if you ignore the packages (because they don't provide any means of encapsulation and hiding), it doesn't really matter which architectural style you're aspiring to create. If we look back at the example UML diagrams, the Java packages become an irrelevant detail if all of the types are marked as `public`. In essence, all four architectural approaches presented earlier in this chapter are exactly the same when we overuse this designation ([Figure 34.7](#)).

Take a close look at the arrows between each of the types in [Figure 34.7](#): They're all identical regardless of which architectural approach you're trying to adopt. Conceptually the approaches are very different, but syntactically they are identical. Furthermore, you could argue that when you make all of the types `public`, what you really have are just four ways to describe a traditional horizontally layered architecture. This is a neat trick, and of course nobody would ever make all of their Java types `public`. Except when they do. And I've seen it.

The access modifiers in Java are not perfect,⁸ but ignoring them is just asking for trouble. The way Java types are placed into packages can actually make a huge difference to how accessible (or inaccessible) those types can be when Java's access modifiers are applied appropriately. If I bring the packages back and mark (by graphically fading) those types where the access modifier can be made more restrictive, the picture becomes pretty interesting ([Figure 34.8](#)).

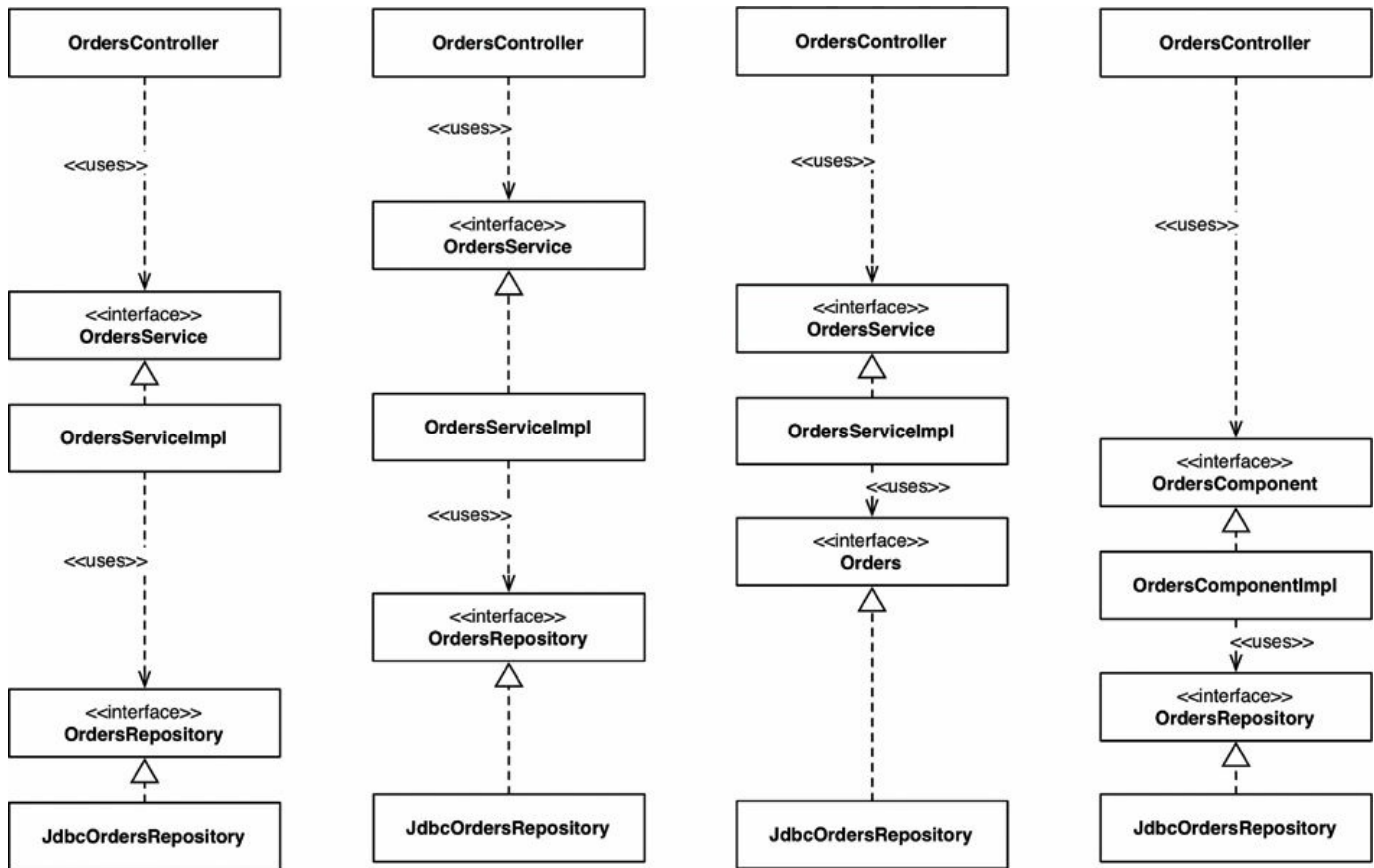


Figure 34.7 All four architectural approaches are the same

Moving from left to right, in the “package by layer” approach, the `OrdersService` and `OrdersRepository` interfaces need to be `public`, because they have inbound dependencies from classes outside of their defining package. In contrast, the implementation classes (`OrdersServiceImpl` and `JdbcOrdersRepository`) can be made more restrictive (package protected). Nobody needs to know about them; they are an implementation detail.

In the “package by feature” approach, the `OrdersController` provides the sole entry point into the package, so everything else can be made package protected. The big caveat here is that nothing else in the code base, outside of this package, can access information related to orders unless they go through the controller. This may or may not be desirable.

In the ports and adapters approach, the `OrdersService` and `Orders` interfaces have inbound dependencies from other packages, so they need to be made `public`. Again, the implementation classes can be made package protected and dependency injected at runtime.

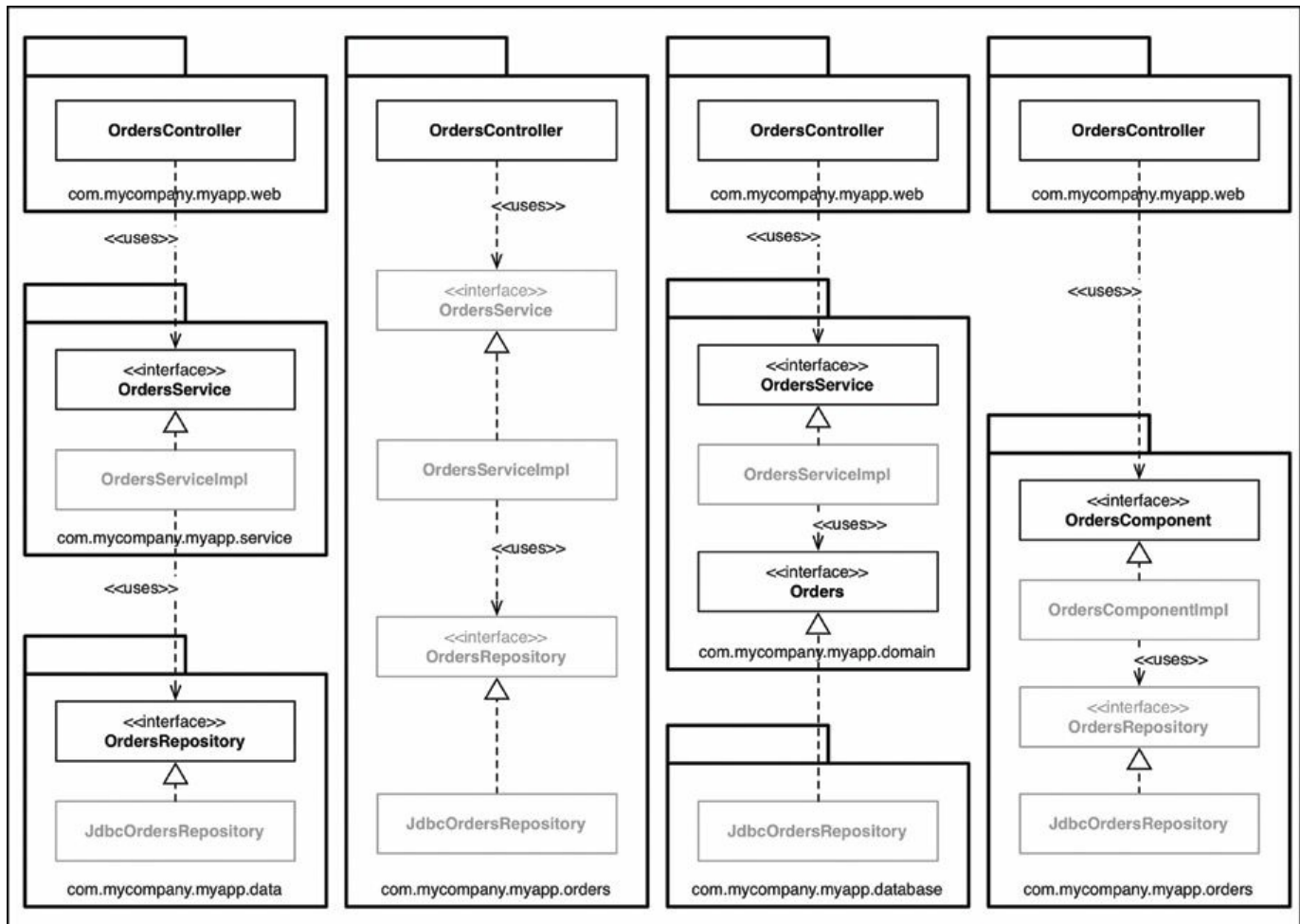


Figure 34.8 Grayed-out types are where the access modifier can be made more restrictive

Finally, in the “package by component” approach, the `OrdersComponent` interface has an inbound dependency from the controller, but everything else can be made package protected. The fewer `public` types you have, the smaller the number of potential dependencies. There’s now no way⁹ that code outside this package can use the `OrdersRepository` interface or implementation directly, so we can rely on the compiler to enforce this architectural principle. You can do the same thing in .NET with the `internal` keyword, although you would need to create a separate assembly for every component.

Just to be absolutely clear, what I’ve described here relates to a monolithic application, where all of the code resides in a single source code tree. If you are building such an application (and many people are), I would certainly encourage you to lean on the compiler to enforce your architectural principles, rather than relying on self-discipline and post-compilation tooling.

OTHER DECOUPLING MODES

In addition to the programming language you're using, there are often other ways that you can decouple your source code dependencies. With Java, you have module frameworks like OSGi and the new Java 9 module system. With module systems, when used properly, you can make a distinction between types that are `public` and types that are *published*. For example, you could create an `Orders` module where all of the types are marked as `public`, but publish only a small subset of those types for external consumption. It's been a long time coming, but I'm enthusiastic that the Java 9 module system will give us another tool to build better software, and spark people's interest in design thinking once again.

Another option is to decouple your dependencies at the source code level, by splitting code across *different source code trees*. If we take the ports and adapters example, we could have three source code trees:

- The source code for the business and domain (i.e., everything that is independent of technology and framework choices): `OrdersService`, `OrdersServiceImpl`, and `Orders`
- The source code for the web: `OrdersController`
- The source code for the data persistence: `JdbcOrdersRepository`

The latter two source code trees have a compile-time dependency on the business and domain code, which itself doesn't know anything about the web or the data persistence code. From an implementation perspective, you can do this by configuring separate modules or projects in your build tool (e.g., Maven, Gradle, MSBuild). Ideally you would repeat this pattern, having a separate source code tree for each and every component in your application. This is very much an idealistic solution, though, because there are real-world performance, complexity, and maintenance issues associated with breaking up your source code in this way.

A simpler approach that some people follow for their ports and adapters code is to have just two source code trees:

- Domain code (the “inside”)
- Infrastructure code (the “outside”)

This maps on nicely to the diagram ([Figure 34.9](#)) that many people use to summarize the ports and adapters architecture, and there is a compile-time dependency from the

infrastructure to the domain.

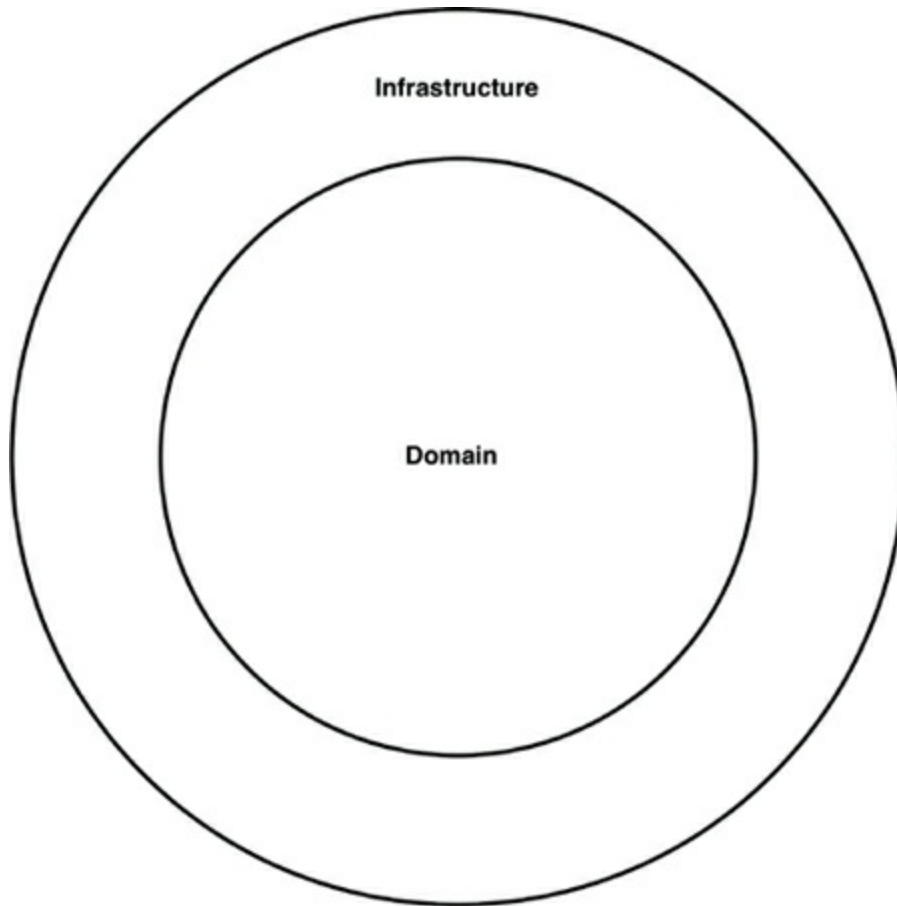


Figure 34.9 Domain and infrastructure code

This approach to organizing source code will also work, but be aware of the potential trade-off. It's what I call the "Périphérique anti-pattern of ports and adapters." The city of Paris, France, has a ring road called the Boulevard Périphérique, which allows you to circumnavigate Paris without entering the complexities of the city. Having all of your infrastructure code in a single source code tree means that it's potentially possible for infrastructure code in one area of your application (e.g., a web controller) to directly call code in another area of your application (e.g., a database repository), without navigating through the domain. This is especially true if you've forgotten to apply appropriate access modifiers to that code.

CONCLUSION: THE MISSING ADVICE

The whole point of this chapter is to highlight that your best design intentions can be destroyed in a flash if you don't consider the intricacies of the implementation

strategy. Think about how to map your desired design on to code structures, how to organize that code, and which decoupling modes to apply during runtime and compile-time. Leave options open where applicable, but be pragmatic, and take into consideration the size of your team, their skill level, and the complexity of the solution in conjunction with your time and budgetary constraints. Also think about using your compiler to help you enforce your chosen architectural style, and watch out for coupling in other areas, such as data models. The devil is in the implementation details.

1. This is arguably a horrible way to name a class, but as we'll see later, perhaps it doesn't really matter.
2. <https://martinfowler.com/bliki/PresentationDomainDataLayering.html>.
3. This benefit is much less relevant with the navigation facilities of modern IDEs, but it seems there has been a renaissance moving back to lightweight text editors, for reasons I am clearly too old to understand.
4. My first job after graduating from university in 1996 was building client-server desktop applications with a technology called PowerBuilder, a super-productive 4GL that excelled at building database-driven applications. A couple of years later, I was building client-server applications with Java, where we had to build our own database connectivity (this was pre-JDBC) and our own GUI toolkits on top of AWT. That's "progress" for you!
5. In the *Command Query Responsibility Segregation* pattern, you have separate patterns for updating and reading data.
6. <http://www.laputan.org/mud/>
7. See <https://www.structurizr.com/help/c4> for more information.
8. In Java, for example, although we tend to think of packages as being hierarchical, it's not possible to create access restrictions based on a package and subpackage relationship. Any hierarchy that you create is in the name of those packages, and the directory structure on disk, only.
9. Unless you cheat and use Java's reflection mechanism, but please don't do that!

VII

Appendix

A ARCHITECTURE ARCHAEOLOGY



To unearth the principles of good architecture, let's take a 45-year journey through some of the projects I have worked on since 1970. Some of these projects are interesting from an architectural point of view. Others are interesting because of the lessons learned and because of how they fed into subsequent projects.

This appendix is somewhat autobiographical. I've tried to keep the discussion relevant to the topic of architecture; but, as in anything autobiographical, other factors sometimes intrude. ; -)

UNION ACCOUNTING SYSTEM

In the late 1960s, a company by the name of ASC Tabulating signed a contract with

Local 705 of the Teamsters Union to provide an accounting system. The computer ASC chose to implement this system on was a GE Datanet 30, as shown in [Figure A.1](#).



Figure A.1 GE Datanet 30

Courtesy Ed Thelen, ed-thelen.org

As you can see from the picture, this was a huge¹ machine. It filled a room, and the room needed strict environmental controls.

This computer was built in the days before integrated circuits. It was built out of discrete transistors. There were even some vacuum tubes in it (albeit only in the sense amplifiers of the tape drives).

By today's standards the machine was huge, slow, small, and primitive. It had $16K \times 18$ bits of core, with a cycle time of about 7 microseconds.² It filled a big, environmentally controlled room. It had 7 track magnetic tape drives and a disk drive with a capacity of 20 megabytes or so.

That disk was a monster. You can see it in the picture in [Figure A.2](#)—but that doesn't quite give you the scale of the beast. The top of that cabinet was over my head. The platters were 36 inches in diameter, and $\frac{3}{8}$ of an inch thick. One of the platters is pictured in [Figure A.3](#).

Now count the platters in that first picture. There are more than a dozen. Each one had its own individual seek arm that was driven by pneumatic actuators. You could watch those seek heads move across the platters. The seek time was probably about

half a second to a second.

When this beast was turned on, it sounded like a jet engine. The floor would rumble and shake until it got up to speed.³



Figure A.2 The data storage unit with its platters

Courtesy Ed Thelen, ed-thelen.org

The great claim to fame of the Datanet 30 was its capability to drive a large number of asynchronous terminals at relatively high speed. That's exactly what ASC needed.

ASC was based in Lake Bluff, Illinois, 30 miles north of Chicago. The Local 705 office was in downtown Chicago. The union wanted a dozen or so of their data entry clerks to use CRT⁴ terminals ([Figure A.4](#)) to enter data into the system. They would print reports on ASR35 teletypes ([Figure A.5](#)).

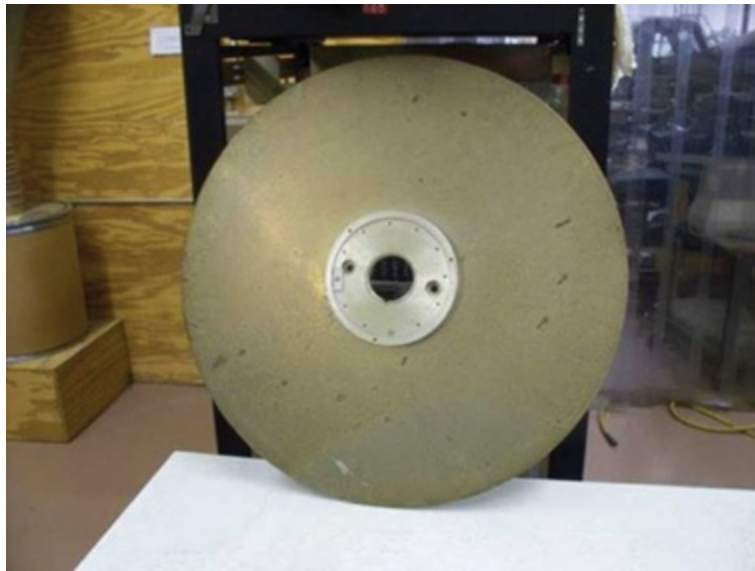


Figure A.3 One platter of that disk: 3/8 inch thick, 36 inches in diameter

Courtesy, Ed Thelen, ed-thelen.org

The CRT terminals ran at 30 characters per second. This was a pretty good rate for the late 1960s because modems were relatively unsophisticated in those days.

ASC leased a dozen or so dedicated phone lines and twice that number of 300 baud modems from the phone company to connect the Datanet 30 to these terminals.

These computers did not come with operating systems. They didn't even come with file systems. What you got was an assembler.

If you needed to store data on the disk, you stored data on the disk. Not in a file. Not in a directory. You figured out which track, platter, and sector to put the data into, and then you operated the disk to put the data there. Yes, that means we wrote our own disk driver.



Figure A.4 Datapoint CRT terminal

Courtesy of Bill Degnan, vintagecomputer.net

The Union Accounting system had three kinds of records: Agents, Employers, and Members. The system was a CRUD system for these records, but also included operations for posting dues, computing changes in the general ledger, and so on.

The original system was written in assembler by a consultant who somehow managed to cram the whole thing into 16K.

As you might imagine, that big Datanet 30 was an expensive machine to operate and maintain. The software consultant who kept the software running was expensive, too. What's more, minicomputers were becoming popular, and were much cheaper.



Figure A.5 ASR35 teletype

Joe Mabel, with permission

In 1971, when I was 18 years old, ASC hired me and two of my geeky friends to replace the whole union accounting system with one that was based on a Varian 620/f minicomputer ([Figure A.6](#)). The computer was cheap. We were cheap. So it seemed like a good deal for ASC.

The Varian machine had a 16-bit bus and 32K * 16 core memory. It had a cycle time of about 1 microsecond. It was much more powerful than the Datamet 30. It used IBM's wildly successful 2314 disk technology, allowing us to store 30 megabytes on platters that were only 14 inches in diameter and could not explode through concrete block walls!

Of course, we still had no operating system. No file system. No high-level language. All we had was an assembler. But we made do.

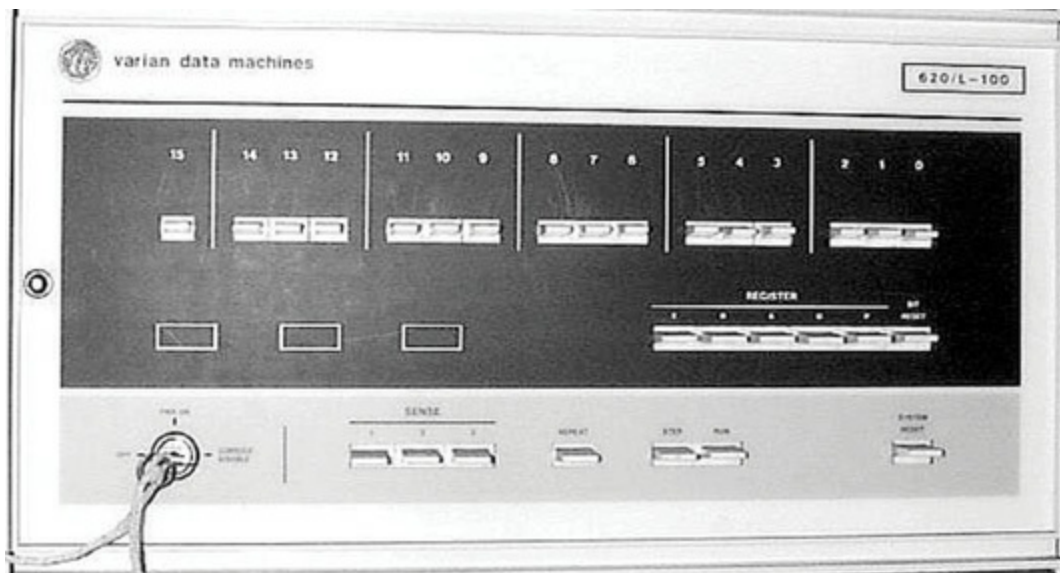


Figure A.6 Varian 620/f minicomputer

The Minicomputer Orphanage

Instead of trying to cram the whole system into 32K, we created an overlay system. Applications would be loaded from disk into a block of memory dedicated to overlays. They would be executed in that memory, and preemptively swapped back out onto disk, with their local RAM, to allow other programs to execute.

Programs would get swapped into the overlay area, execute enough to fill the output

buffers, and then get swapped out so that another program could be swapped in.

Of course, when your UI runs at 30 characters per second, your programs spend a lot of time waiting. We had plenty of time to swap the programs in and off the disk to keep all of the terminals running as fast as they could go. Nobody ever complained of response time issues.

We wrote a preemptive supervisor that managed the interrupts and IO. We wrote the applications; we wrote the disk drivers, the terminal drivers, the tape drivers, and everything else in that system. There was not a single bit in that system that we did not write. Though it was a struggle involving far too many 80-hour weeks, we got the beast up and running in a matter of 8 or 9 months.

The architecture of the system was simple ([Figure A.7](#)). When an application was started, it would generate output until its particular terminal buffer was full. Then the supervisor would swap the application out, and swap a new application in. The supervisor would continue to dribble out the contents of the terminal buffer at 30 cps until it was nearly empty. Then it would swap the application back in to fill the buffer again.

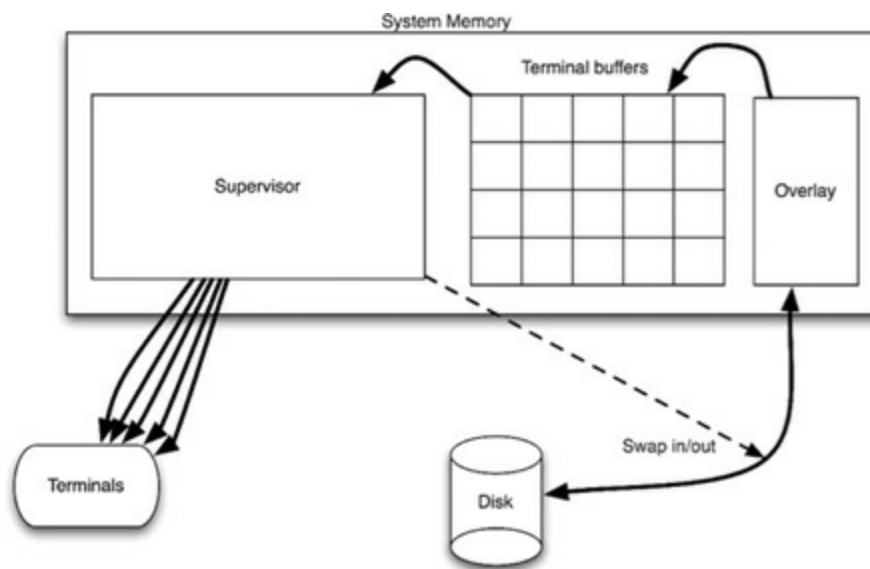


Figure A.7 The system architecture

There are two boundaries in this system. The first is the character output boundary. The applications had no idea that their output was going to a 30-cps terminal. Indeed, the character output was entirely abstract from the applications' point of view. The applications simply passed strings to the supervisor, and the supervisor took care of loading the buffers, sending the characters to the terminals, and swapping the

applications in and out of memory.

This boundary was dependency normal—that is, dependencies pointed *with* the flow of control. The applications had compile-time dependencies on the supervisor, and the flow of control passed from the applications to the supervisor. The boundary prevented the applications from knowing which kind of device the output was going to.

The second boundary was dependency inverted. The supervisor could start the applications, but had no compile-time dependencies upon them. The flow of control passed from the supervisor to the applications. The polymorphic interface that inverted the dependency was simply this: Every application was started by jumping to the exact same memory address within the overlay area. The boundary prevented the supervisor from knowing anything about the applications other than the starting point.

LASER TRIM

In 1973, I joined a company in Chicago called Teradyne Applied Systems (TAS). This was a division of Teradyne Inc., which was headquartered in Boston. Our product was a system that used relatively high-powered lasers to trim electronic components to very fine tolerances.

Back in those days, manufacturers would silk-screen electronic components onto ceramic substrates. Those substrates were on the order of 1 inch square. The components were typically resistors—devices that resist the flow of current.

The resistance of a resistor depends on a number of factors, including its composition and its geometry. The wider the resistor, the less resistance it has.

Our system would position the ceramic substrate in a harness that had probes that made contact with the resistors. The system would measure the resistance of the resistors, and then use a laser to burn off parts of the resistor, making it thinner and thinner until it reached the desired resistance value within a tenth of a percent or so.

We sold these systems to manufacturers. We also used some in-house systems to trim relatively small batches for small manufacturers.

The computer was an M365. This was in the days when many companies built their own computers: Teradyne built the M365 and supplied it to all its divisions. The

M365 was an enhanced version of a PDP-8—a popular minicomputer of the day.

The M365 controlled the positioning table, which moved the ceramic substrates under the probes. It controlled the measurement system and the laser. The laser was positioned using *X-Y* mirrors that could rotate under program control. The computer could also control the power setting of the laser.

The development environment of the M365 was relatively primitive. There was no disk. Mass storage was on tape cartridges that looked like old 8-track audio tape cassettes. The tapes and drives were made by Tri-Data.

Like the 8-track audio cassettes of the day, the tape was oriented in a loop. The drive moved the tape in only one direction—there was no rewind! If you wanted to position the tape at the beginning, you had to send it forward to its “load point.”

The tape moved at a speed of approximately 1 foot per second. Thus, if the tape loop was 25 feet long, it could take as long as 25 seconds to send it to the load point. For this reason Tridata made cartridges in several lengths, ranging from 10 feet to 100 feet.

The M365 had a button on the front that would load memory with a little bootstrap program and execute it. This program would read the first block of data from the tape, and execute that. Typically this block held a loader that loaded the operating system that lived on the rest of the tape.

The operating system would prompt the user for the name of a program to run. Those programs were stored on the tape, just after the operating system. We would type in the name of the program—for example, the ED-402 Editor—and the operating system would search the tape for that program, load it, and execute it.

The console was an ASCII CRT with green phosphors, 72 characters wide⁵ by 24 lines. The characters were all uppercase.

To edit a program, you would load the ED-402 Editor, and then insert the tape that held your source code. You would read one tape block of that source code into memory, and it would be displayed on the screen. The tape block might hold 50 lines of code. You would make your edits by moving the cursor around on the screen and typing in a manner similar to `vi`. When you were done, you would write that block onto a different tape, and read the next block from the source tape. You kept on doing this until you were done.

There was no scrolling back to previous blocks. You edited your program in a straight line, from beginning to end. Going back to the beginning forced you to finish copying the source code onto the output tape and then start a new editing session on that tape. Perhaps not surprisingly, given these constraints, we printed our programs out on paper, marked all the edits by hand in red ink, and then edited our programs block by block by consulting our markups on the listing.

Once the program was edited, we returned to the OS and invoked the assembler. The assembler read the source code tape, and wrote a binary tape, while also producing a listing on our data products line printer.

The tapes weren't 100% reliable, so we always wrote two tapes at the same time. That way, at least one of them had a high probability of being free of errors.

Our program was approximately 20,000 lines of code, and took nearly 30 minutes to compile. The odds that we would get a tape read error during that time were roughly 1 in 10. If the assembler got a tape error, it would ring the bell on the console and then start printing a stream of errors on the printer. You could hear this maddening bell all across the lab. You could also hear the cursing of the poor programmer who just learned that the 30-minute compile needed to start over.

The architecture of the program was typical for those days. There was a Master Operating Program, appropriately called "the MOP." Its job was to manage basic IO functions and provide the rudiments of a console "shell." Many of the divisions of Teradyne shared the MOP source code, but each had forked it for its own uses. Consequently, we would send source code updates around to each other in the form of marked-up listings that we would then integrate manually (and very carefully).

A special-purpose utility layer controlled the measurement hardware, the positioning tables, and the laser. The boundary between this layer and the MOP was muddled at best. While the utility layer called the MOP, the MOP had been specifically modified for that layer, and often called back into it. Indeed, we didn't really think of these two as separate layers. To us, it was just some code that we added to the MOP in a highly coupled way.

Next came the isolation layer. This layer provided a virtual machine interface for the application programs, which were written in a completely different domain-specific data-driven language (DSL). The language had operations for moving the laser, moving the table, making cuts, making measurements, and so on. Our customers would write their laser trimming application programs in this language, and the

isolation layer would execute them.

This approach was not intended to create a machine-independent laser trim language. Indeed, the language had many idiosyncrasies that were deeply coupled to the layers below. Rather, this approach gave the application programmers a “simpler” language than M356 assembler in which to program their trim jobs.

Trim jobs could be loaded from tape and executed by the system. Essentially, our system was an operating system for trim applications.

The system was written in M365 assembler and compiled in a single compilation unit that produced absolute binary code.

The boundaries in this application were soft at best. Even the boundary between the system code and the applications written in the DSL was not well enforced. There were couplings everywhere.

But that was typical of software in the early 1970s.

ALUMINUM DIE-CAST MONITORING

In the middle of the 1970s, while OPEC was placing an embargo on oil, and gasoline shortages were causing angry drivers to get into fights at gas stations, I began working at Outboard Marine Corporation (OMC). This is the parent company of Johnson Motors and Lawnboy lawnmowers.

OMC maintained a huge facility in Waukegan, Illinois, for creating die-cast aluminum parts for all of the company’s motors and products. Aluminum was melted down in huge furnaces, and then carried in large buckets to dozens upon dozens of individually operated aluminum die-cast machines. Each machine had a human operator responsible for setting the molds, cycling the machine, and extracting the newly cast parts. These operators were paid based on how many parts they produced.

I was hired to work on a shop-floor automation project. OMC had purchased an IBM System/7—which was IBM’s answer to the minicomputer. They tied this computer to all the die-cast machines on the floor, so that we could count, and time, the cycles of each machine. Our role was to gather all that information and present it on 3270 green-screen displays.

The language was assembler. And, again, every bit of code that executed in this

computer was code that we wrote. There was no operating system, no subroutine libraries, and no framework. It was just raw code.

It was also interrupt-driven real-time code. Every time a die-cast machine cycled, we had to update a batch of statistics, and send messages to a great IBM 370 in-the-sky, running a CICS-COBOL program that presented those statistics on the green screens.

I hated this job. Oh, boy, did I. Oh, the *work* was *fun*! But the culture ... Suffice it to say that I was *required* to wear a tie.

Oh, I tried. I really did. But I was clearly unhappy working there, and my colleagues knew it. They knew it because I couldn't remember critical dates or manage to get up early enough to attend important meetings. This was the only programming job I was ever fired from—and I deserved it.

From an architectural point of view, there's not a lot to learn here except for one thing. The System/7 had a very interesting instruction called *set program interrupt* (SPI). This allowed you to trigger an interrupt of the processor, allowing it to handle any other queued lower-priority interrupts. Nowadays, in Java we call this

`Thread.yield()`.

4-TEL

In October 1976, having been fired from OMC, I returned to a different division of Teradyne—a division I would stay with for 12 years. The product I worked on was named 4-TEL. Its purpose was to test every telephone line in a telephone service area, every night, and produce a report of all lines requiring repair. It also allowed telephone test personnel to test specific telephone lines in detail.

This system started its life with the same kind of architecture as the Laser Trim system. It was a monolithic application written in assembly language without any significant boundaries. But at the time I joined the company, that was about to change.

The system was used by testers located in a service center (SC). A service center covered many central offices (CO), each of which could handle as many as 10,000 phone lines. The dialing and measurement hardware had to be located inside the CO. So that's where the M365 computers were put. We called those computers the central office line testers (COLTs). Another M365 was placed at the SC; it was

called the service area computer (SAC). The SAC had several modems that it could use to dial up the COLTs and communicate at 300 baud (30 cps).

At first, the COLT computers did everything, including all the console communication, menus, and reports. The SAC was just a simple multiplexor that took the output from the COLTs and put it on a screen.

The problem with this setup was that 30 cps is really slow. The testers didn't like watching the characters trickle across the screen, especially since they were only interested in a few key bits of data. Also, in those days, the core memory in the M365 was expensive, and the program was big.

The solution was to separate the part of the software that dialed and measured lines from the part that analyzed the results and printed the reports. The latter would be moved into the SAC, and the former would remain behind in the COLTs. This would allow the COLT to be a smaller machine, with much less memory, and would greatly speed up the response at the terminal, since the reports would be generated in the SAC.

The result was remarkably successful. Screen updates were very fast (once the appropriate COLT had been dialed), and the memory footprint of the COLTs shrank a lot.

The boundary was very clean and highly decoupled. Very short packets of data were exchanged between the SAC and COLT. These packets were a very simple form of DSL, representing primitive commands like "DIAL XXXX" or "MEASURE."

The M365 was loaded from tape. Those tape drives were expensive and weren't very reliable—especially in the industrial environment of a telephone central office. Also, the M365 was an expensive machine relative to the rest of the electronics within the COLT. So we embarked upon a project to replace the M365 with a microcomputer based on an 8085 μ processor.

The new computer was composed of a processor board that held the 8085, a RAM board that held 32K of RAM, and three ROM boards that held 12K of read-only memory apiece. All these boards fit into the same chassis as the measurement hardware, thereby eliminating the bulky extra chassis that had housed the M365.

The ROM boards held 12 Intel 2708 EPROM (Erasable Programmable Read-Only Memory) chips.⁶ [Figure A.8](#) shows an example of such a chip. We loaded those chips with software by inserting them into special devices called PROM burners that

were driven by our development environment. The chips could be erased by exposing them to high-intensity ultraviolet light.⁷

My buddy CK and I translated the M365 assembly language program for the COLT into 8085 assembly language. This translation was done by hand and took us about 6 months. The end result was approximately 30K of 8085 code.

Our development environment had 64K of RAM and no ROM, so we could quickly download our compiled binaries into RAM and test them.

Once we got the program working, we switched to using the EPROMs. We burned 30 chips and inserted them into just the right slots in the three ROM boards. Each chip was labeled so we could tell which chip went into which slot.

The 30K program was a single binary, 30K long. To burn the chips, we simply divided that binary image into 30 different 1K segments, and burned each segment onto the appropriately labeled chip.

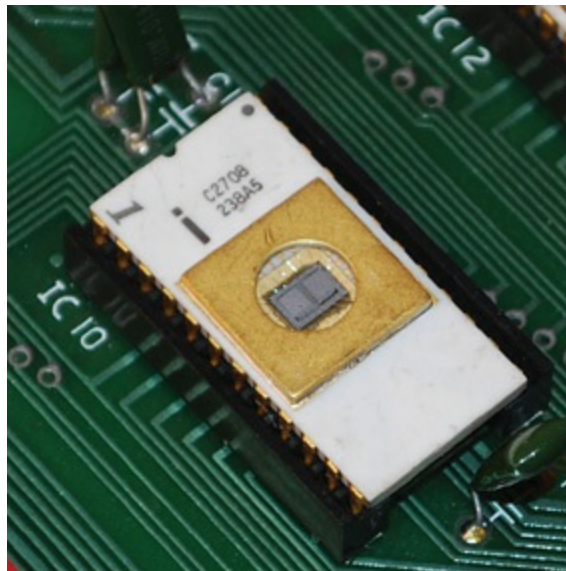


Figure A.8 EPROM chip

This worked very well, and we began to mass-produce the hardware and deploy the system into the field.

But software is soft.⁸ Features needed to be added. Bugs needed to be fixed. And as the installed base grew, the logistics of updating the software by burning 30 chips per installation, and having field service people replace all 30 chips at each site became a nightmare.

There were all kinds of problems. Sometimes chips would be mislabeled, or the labels would fall off. Sometimes the field service engineer would mistakenly replace the wrong chip. Sometimes the field service engineer would inadvertently break a pin off one of the new chips. Consequently, the field engineers had to carry extras of all 30 chips with them.

Why did we have to change all 30 chips? Every time we added or removed code from our 30K executable, it changed the addresses in which each instruction was loaded. It also changed the addresses of the subroutines and functions that we called. So every chip was affected, no matter how trivial the change.

One day, my boss came to me and asked me to solve that problem. He said we needed a way to make a change to the firmware without replacing all 30 chips every time. We brainstormed this issue for a while, and then embarked upon the “Vectorization” project. It took me three months.

The idea was beautifully simple. We divided the 30K program into 32 independently compilable source files, each less than 1K. At the beginning of each source file, we told the compiler in which address to load the resulting program (e.g., ORG C400 for the chip that was to be inserted into the C4 position).

Also at the beginning of each source file, we created a simple, fixed-size data structure that contained all the addresses of all the subroutines on that chip. This data structure was 40 bytes long, so it could hold no more than 20 addresses. This meant that no chip could have more than 20 subroutines.

Next, we created a special area in RAM known as the vectors. It contained 32 tables of 40 bytes—exactly enough RAM to hold the pointers at the start of each chip.

Finally, we changed every call to every subroutine on every chip into an indirect call through the appropriate RAM vector.

When our processor booted, it would scan each chip and load the vector table at the start of each chip into the RAM vectors. Then it would jump into the main program.

This worked very well. Now, when we fixed a bug, or added a feature, we could simply recompile one or two chips, and send just those chips to the field service engineers.

We had made the chips *independently deployable*. We had invented polymorphic dispatch. We had invented objects.

This was a plugin architecture, quite literally. We plugged those chips in. We eventually engineered it so that a feature could be installed into our products by plugging the chip with that feature into one of the open chip sockets. The menu control would automatically appear, and the binding into the main application would happen automatically.

Of course, we didn't know about object-oriented principles at the time, and we knew nothing about separating user interface from business rules. But the rudiments were there, and they were very powerful.

One unexpected side benefit of the approach was that we could patch the firmware over a dial-up connection. If we found a bug in the firmware, we could dial up our devices and use the on-board monitor program to alter the RAM vector for the faulty subroutine to point to a bit of empty RAM. Then we'd enter the repaired subroutine into that RAM area, by typing it in machine code, in hexadecimal.

This was a great boon to our field service operation, and to our customers. If they had a problem, they didn't need us to ship new chips and schedule an urgent field service call. The system could be patched, and a new chip could be installed at the next regularly scheduled maintenance visit.

THE SERVICE AREA COMPUTER

The 4-TEL service area computer (SAC) was based on an M365 minicomputer. This system communicated with all the COLTs out in the field, through either dedicated or dial-up modems. It would command those COLTs to measure telephone lines, would receive back the raw results, and would then perform a complex analysis of those results to identify and locate any faults.

DISPATCH DETERMINATION

One of the economic foundations for this system was based on the correct allocation of repair craftsmen. Repair craft were separated, by union rules, into three categories: central office, cable, and drop. CO craftsmen fixed problems inside the central office. Cable craftsmen fixed problems in the cable plant that connected the CO to the customer. Drop craftsmen fixed problems inside the customer's premises, and in the lines connecting the external cable to that premises (the "drop").

When a customer complained about a problem, our system could diagnose that